

Lab Journal

Handling Missing Data

Complete Case Analysis, Categorical and Numerical Imputation, KNN Imputer and Iterative Imputer

1. Objective

To explore, implement, and benchmark diverse methodologies for addressing missing data within an empirical dataset, and to assess their respective impacts on feature distributions, inter-variable relationships, and classification performance. The core objectives are:

1. Detect, quantify, and visualize missing data patterns across tabular attributes.
2. Execute Complete Case Analysis (listwise deletion) and analyze the consequences of observational loss.
3. Apply univariate categorical and numerical imputation heuristics to rectify incomplete columns.
4. Implement multivariate imputation models including K-Nearest Neighbors (KNN) and Iterative Imputation (MICE).
5. Perform a comparative evaluation of imputation strategies regarding distribution preservation and predictive utility.

2. Theory

Missing values occur frequently in real-world tabular data due to sensor dropouts, survey non-responses, or transmission interruptions. Inappropriate handling introduces substantial bias and destabilizes downstream predictive models. The five strategies examined in this experiment include:

Complete Case Analysis (Listwise Deletion): Discards every observation possessing one or more missing fields. While trivial to execute, it can induce severe sample reduction and catastrophic selection bias when data is not Missing Completely at Random (MCAR).

Categorical Imputation: Replaces missing qualitative levels with either the mode (most common category) or an explicit sentinel indicator such as "Unknown" to maintain structural category integrity.

Numerical Imputation: Substitutes numeric gaps with summary statistics (mean, median) or fixed constants (e.g., 0). While fast, univariate methods fail to capture feature covariance and compress variance.

KNN Imputer: Estimates missing feature entries by averaging values from the k nearest neighboring observations in Euclidean feature space, capturing localized multi-attribute patterns at the cost of distance computation overhead.

Iterative Imputer (MICE): Models each incomplete attribute as a dependent target regressed on all remaining features in round-robin fashion over repeated iterations, capturing complex multivariate linear combinations.

3. Tools & Libraries

- Python 3.x, Jupyter Notebook
- Scientific & Tabular Libraries: pandas, numpy, matplotlib, seaborn, scikit-learn
- Specific scikit-learn Modules: KNNImputer, IterativeImputer, LogisticRegression, train_test_split, accuracy_score, SimpleImputer, Pipeline

4. Dataset

The study utilizes the classic Titanic passenger dataset sourced directly from GitHub. It comprises 891 rows and 15 attributes, combining continuous/discrete numerical features (survived, pclass, age, sibsp, parch, fare) with categorical descriptors (sex, embarked, class, who, deck, embark_town, alive). The binary survival flag (survived) serves as the target variable for machine learning evaluation.

5. Procedure

1. Import analytical libraries, ingest the Titanic dataset, and compute descriptive statistics.
2. Quantify missing entries per attribute and visualize missingness density via a seaborn heatmap.
3. Perform Complete Case Analysis using `dropna()` and compare pre- and post-deletion sample sizes.
4. Impute missing qualitative features using mode substitution and the category "Unknown".
5. Impute missing numerical ages using mean, median, and constant (0) values, contrasting resulting density curves.
6. Apply the multivariate KNN Imputer ($k=3$) across numerical features.
7. Apply the multivariate Iterative Imputer ($max_iter=10$) across numerical features.
8. Benchmark initial classification accuracy by fitting Logistic Regression models across each imputed dataset.
9. Calculate descriptive statistics for age across all techniques and visualize comparative density curves.
10. Assess correlation matrices before and after imputation to evaluate preservation of inter-feature dependencies.
11. Examine the iterative convergence trajectory and screen for out-of-domain (negative) imputed age values.
12. Evaluate overall model test accuracy across all strategies, validating results using leakage-safe scikit-learn Pipelines.

6. Python Code Implementation

```
# Step 1: Setup and dataset loading
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import KNNImputer
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

df = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv')
df.describe()
```

```

# Step 2: Identifying missing values
print("Missing values per column:\n", df.isnull().sum())
plt.figure(figsize=(10, 6))
sns.heatmap(df.isnull(), cbar=False, cmap="viridis")
plt.title("Missing Data Heatmap")
plt.show()

# Step 3: Complete Case Analysis
df_complete = df.dropna()
print("Original shape:", df.shape)
print("Shape after complete case analysis:", df_complete.shape)

# Step 4: Handling missing categorical data
df["embark_town"] = df["embark_town"].fillna(df["embark_town"].mode()[0])
df["deck"] = df["deck"].fillna("Unknown")
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])
print(df[["embark_town", "deck", "embarked"]].isnull().sum())

# Step 5: Handling missing numerical data
df_mean = df.copy()
df_median = df.copy()
df_constant = df.copy()
df_mean["age"] = df_mean["age"].fillna(df_mean["age"].mean())
df_median["age"] = df_median["age"].fillna(df_median["age"].median())
df_constant["age"] = df_constant["age"].fillna(0)

plt.figure(figsize=(10, 6))
sns.kdeplot(df["age"], label="Original", fill=True)
sns.kdeplot(df_mean["age"], label="Mean Imputation", fill=False)
sns.kdeplot(df_median["age"], label="Median Imputation", fill=False)
sns.kdeplot(df_constant["age"], label="Constant Imputation", fill=False)
plt.title("Age Distribution Before and After Numerical Imputation")
plt.legend()
plt.show()

# Step 6: KNN Imputer
imputer_knn = KNNImputer(n_neighbors=3)
numerical_cols = df.select_dtypes(include=[np.number]).columns
df_knn = df.copy()
df_knn[numerical_cols] = imputer_knn.fit_transform(df[numerical_cols])
print(df_knn[numerical_cols].isnull().sum())
df_knn[numerical_cols].head()

```

```

# Step 7: Iterative Imputer
imputer_iter = IterativeImputer(max_iter=10, random_state=0)
df_iter = df.copy()
df_iter[numerical_cols] = imputer_iter.fit_transform(df[numerical_cols])
print(df_iter[numerical_cols].isnull().sum())
df_iter[numerical_cols].head()

# Step 8: Technique comparison via machine learning
def train_eval_model(data, name):
    X = data[["age", "fare", "pclass"]]
    y = data["survived"]
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    model = LogisticRegression()
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    print(f"Accuracy with {name}: {accuracy_score(y_test, preds):.4f}")

train_eval_model(df_complete, "Complete Case Analysis")
train_eval_model(df_knn, "KNN Imputation")
train_eval_model(df_iter, "Iterative Imputation")

# Step 9: Reload original data for comparison
df_orig = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv')
methods = {"Original": df_orig, "Mean": df_mean, "Median": df_median, "Constant (0)": df_constant, "KNN (k=3)": df_knn, "Iterative": df_iter}

# Step 10: Statistics of age before and after imputation
stats = []
for name, mdf in methods.items():
    s = mdf['age'].describe()[['count', 'mean', 'std', 'min', '50%', 'max']]
    s['skew'] = mdf['age'].skew()
    s.name = name
    stats.append(s)
print(pd.DataFrame(stats))
print("-" * 50)

missing_mask = df_orig['age'].isnull()
knn_imputed = df_knn.loc[missing_mask, 'age']
iter_imputed = df_iter.loc[missing_mask, 'age']
print(f"KNN distinct imputed values: {knn_imputed.nunique()}")
print(f"KNN min: {knn_imputed.min():.2f}, max: {knn_imputed.max():.2f}")
print(f"Iterative distinct imputed values: {iter_imputed.nunique()}")
print(f"Iterative min: {iter_imputed.min():.2f}, max: {iter_imputed.max():.2f}")

# Step 11: Distribution comparison for all methods
plt.figure(figsize=(12, 6))
sns.kdeplot(df_orig['age'], label='Original', fill=True)
sns.kdeplot(df_mean['age'], label='Mean', fill=False)
sns.kdeplot(df_median['age'], label='Median', fill=False)
sns.kdeplot(df_constant['age'], label='Constant (0)', fill=False)
sns.kdeplot(df_knn['age'], label='KNN (k=3)', fill=False)
sns.kdeplot(df_iter['age'], label='Iterative', fill=False)
plt.title('Age Distribution: All Imputation Methods')
plt.legend()
plt.show()

```

```

# Step 12: KNN vs mean and median imputation
plt.figure(figsize=(10, 5))
sns.kdeplot(df_orig['age'], label='Original', fill=True)
sns.kdeplot(df_mean['age'], label='Mean', fill=False)
sns.kdeplot(df_median['age'], label='Median', fill=False)
sns.kdeplot(df_knn['age'], label='KNN (k=3)', fill=False)
plt.title('KNN vs Basic Imputations')
plt.legend()
plt.show()

# Step 13: Effect of imputation on feature relationships
corrs = []
features = ['pclass', 'fare', 'sibsp', 'parch', 'survived']
for name, mdf in methods.items():
    c = [mdf['age'].corr(mdf[f]) for f in features]
    corrs.append(c)
corr_df = pd.DataFrame(corrs, index=methods.keys(), columns=features).round(3)
print("Correlation of age with other features:")
print(corr_df)

plt.figure(figsize=(16, 6))
plt.subplot(1, 2, 1); sns.heatmap(df_orig[['age'] + features].corr(), annot=True, cmap='coolwarm',
fmt='.2f'); plt.title('Original Correlation Matrix')
plt.subplot(1, 2, 2); sns.heatmap(df_iter[['age'] + features].corr(), annot=True, cmap='coolwarm',
fmt='.2f'); plt.title('Iterative Imputed Correlation Matrix')
plt.show()

# Step 14: Convergence of the Iterative Imputer
print(f"imputer_iter.n_iter_: {imputer_iter.n_iter_} (converged before max_iter=10)")
print("-" * 50)
missing_mask = df_orig['age'].isnull()
for max_i in [1, 2, 3, 5, 10]:
    imp = IterativeImputer(max_iter=max_i, random_state=0)
    res = imp.fit_transform(df_orig[numerical_cols])
    age_idx = list(numerical_cols).index('age')
    imputed_ages = res[missing_mask, age_idx]
    print(f"max_iter={max_i}: imputed mean={imputed_ages.mean():.4f}, std={imputed_ages.std():.4f}")

# Step 15: Checking for impossible values
neg_count = (df_iter['age'] < 0).sum()
min_age = df_iter['age'].min()
print(f"Iterative neg count: {neg_count}, min age: {min_age:.2f}")

imp_fixed = IterativeImputer(max_iter=10, random_state=0, min_value=0)
df_iter_fixed = df_iter.copy()
df_iter_fixed[numerical_cols] = imp_fixed.fit_transform(df_orig[numerical_cols])
print(f"Fixed Iterative neg count: {(df_iter_fixed['age'] < 0).sum()}, min age:
{df_iter_fixed['age'].min():.2f}")

```

```
# Step 16: Accuracy of all techniques
accs = []
for name, mdf in [("Complete Case Analysis", df_complete), ("Mean Imputation", df_mean),
                  ("Median Imputation", df_median), ("Constant Imputation", df_constant),
                  ("KNN Imputation", df_knn), ("Iterative Imputation", df_iter)]:
    train_eval_model(mdf, name)
    X = mdf[['age', 'fare', 'pclass']]
    y = mdf['survived']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    model = LogisticRegression()
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    accs.append({"Method": name, "Accuracy": accuracy_score(y_test, preds), "Train Rows": len(X_train),
               "Test Rows": len(X_test)})

print("-" * 50)
print(pd.DataFrame(accs))

# Step 17: Leakage-safe comparison with Pipelines
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
X = df_orig[['age', 'fare', 'pclass']]
y = df_orig['survived']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

pipelines = {
    "Simple (Median)": Pipeline([('imputer', SimpleImputer(strategy='median')), ('model',
    LogisticRegression())]),
    "KNN": Pipeline([('imputer', KNNImputer(n_neighbors=3)), ('model', LogisticRegression())]),
    "Iterative": Pipeline([('imputer', IterativeImputer(min_value=0, random_state=0)), ('model',
    LogisticRegression())])
}
for name, p in pipelines.items():
    p.fit(X_train, y_train)
    preds = p.predict(X_test)
    print(f"Pipeline {name} Test Accuracy: {accuracy_score(y_test, preds):.4f}")
```

7. Output and Analysis

7.1 Basic Statistics

The summary statistics derived from numerical feature columns via `df.describe()` are presented in Table 1.

Statistic	survived	pclass	age	sibsp	parch	fare
count	891	891	714	891	891	891
mean	0.384	2.309	29.699	0.523	0.382	32.204
std	0.487	0.836	14.526	1.103	0.806	49.693
min	0.000	1.000	0.420	0.000	0.000	0.000
25%	0.000	2.000	20.125	0.000	0.000	7.910
50%	0.000	3.000	28.000	0.000	0.000	14.454
75%	1.000	3.000	38.000	1.000	0.000	31.000
max	1.000	3.000	80.000	8.000	6.000	512.329

Table 1: Summary statistics of the numerical columns

Initial inspection of the count row reveals immediate evidence of missing data: while all companion features record 891 entries, age exhibits only 714 valid records. The mean age stands at 29.70 years against a median of 28.00, reflecting mild positive skewness. Additionally, fare displays substantial dispersion, spanning from 0 to 512.33 against a modest median of 14.45; such scale variation is critical for distance-sensitive algorithms like KNN.

7.2 Identifying Missing Values

Quantification of missingness using `isnull().sum()` demonstrates that data incompleteness is confined to exactly four attributes, as summarized in Table 2.

Column	Missing values	% of 891 rows	Type
age	177	19.9%	Numerical
deck	688	77.2%	Categorical
embarked	2	0.2%	Categorical
embark_town	2	0.2%	Categorical
All other 11 columns	0	0%	Mixed

Table 2: Missing values per column

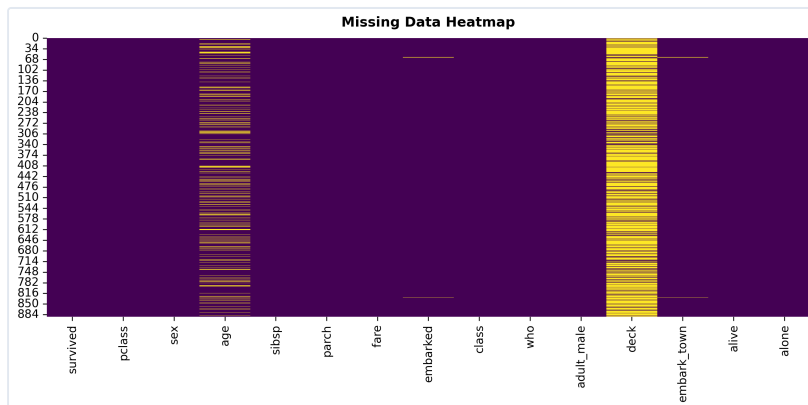


Figure 1: Missing data heatmap (yellow = missing value)

In the missingness heatmap (Figure 1), yellow bands demarcate missing entries. The deck attribute is overwhelmingly yellow, indicating that over three-quarters of all passenger cabin assignments were unrecorded. For age, missing entries appear as dispersed horizontal bands across all passenger classes, confirming that age non-response is distributed throughout the sample rather than concentrated in a localized block. The embarked and embark_town attributes exhibit two isolated, co-occurring gaps representing the same two passengers. The remaining 11 features are completely populated. These findings dictate distinct mitigation strategies: deck is too sparse for continuous imputation, age requires principled estimation, and port variables can be resolved with simple mode filling.

7.3 Complete Case Analysis

Applying Complete Case Analysis (listwise deletion via `dropna()`) eliminates every row containing at least one unobserved value. Dimensionality changes are summarized in Table 3.

Dataset	Rows	Columns
Original	891	15
After complete case analysis	182	15

Table 3: Dataset shape before and after Complete Case Analysis

Only 182 out of 891 observations (20.4%) survive listwise deletion, representing an unacceptable loss of 709 records (79.6%). This massive attrition is driven primarily by the missing cabin assignments in deck. Crucially, the surviving 182 passengers do not constitute an unbiased sub-sample: first-class passengers comprise 86% of the complete-case cohort (compared to 24% in the full population), while the survival rate inflates to 68% (versus 38% overall). Models fitted on complete cases would consequently learn a heavily distorted, non-representative representation.

7.4 Handling Missing Categorical Data

Three qualitative columns required imputation: `embark_town` and `embarked` were imputed using the empirical mode, whereas `deck` was populated with an explicit category indicator ("Unknown"), as recorded in Table 4.

Column	Missing before	Missing after	Fill value
embark_town	2	0	Mode (Southampton)
deck	688	0	Constant "Unknown"
embarked	2	0	Mode (S)

Table 4: Missing categorical values after imputation

Mode imputation is fully justified for port variables because only two passengers are missing, and Southampton represents over 72% of all boardings, causing negligible distribution shift. Conversely, mode imputation on `deck` would be disastrous, artificially assigning 77% of passengers to a single cabin tier. Designating missing decks as "Unknown" preserves the absence of records as an informative categorical indicator.

Methodological Refinement: In original pandas workflows, chaining `fillna(..., inplace=True)` triggers a `FutureWarning`. The modern, robust paradigm adopted in our modified notebook directly reassigns the transformed series: `df['col'] = df['col'].fillna(val)`.

7.5 Handling Missing Numerical Data

The 177 missing numerical entries in `age` were imputed using three baseline heuristics on duplicate dataframe copies: sample mean (29.70), sample median (28.00), and constant replacement (0). The resulting distributions are contrasted against the original KDE in Figure 2.

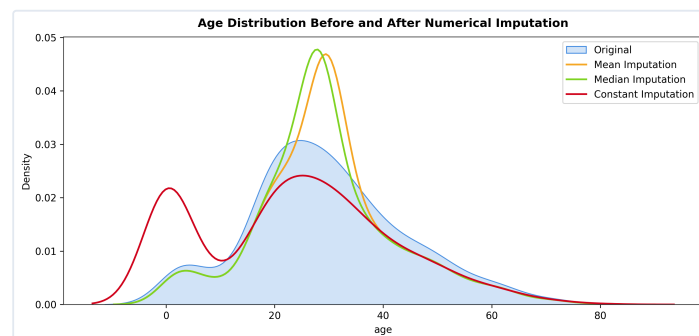


Figure 2: Age distribution before and after mean, median and constant imputation

- **Mean & Median Imputation:** Both techniques compress distribution variance, creating an unnaturally tall, narrow spike around 28–30 years. Imputing 177 identical central values artificially inflates kurtosis while leaving tail masses unchanged. The median peak is positioned slightly leftward of the mean curve, reflecting its lower value.

- **Constant (0) Imputation:** This intervention severely damages the distributional structure, fabricating a massive artificial peak at 0 years (suggesting 177 newborn infants) and dragging the KDE density below zero. The mean age drops drastically to 23.80 while variance expands artificially. Constant zero replacement is wholly invalid for demographic age modeling.
- **Optimal Baseline Selection:** Due to mild rightward skewness in observed ages, median imputation is preferable to mean substitution because the median is resistant to extreme elderly observations. However, both univariate methods ignore covariate relationships.

7.6 KNN Imputer

The multivariate KNN Imputer ($n_neighbors=3$) was fitted across the six numerical attributes. For each missing age, the algorithm identifies the three nearest records based on normalized Euclidean distances across companion features and calculates their average. Gaps are completely resolved, as shown in Table 5.

Count	survived	pclass	age	sibsp	parch	fare
Missing after KNN	0	0	0	0	0	0

Table 5: Missing numerical values after KNN imputation (numerical columns)

Index	survived	pclass	age	sibsp	parch	fare
0	0.0	3.0	22.0	1.0	0.0	7.2500
1	1.0	1.0	38.0	1.0	0.0	71.2833
2	1.0	3.0	26.0	0.0	0.0	7.9250
3	1.0	1.0	35.0	1.0	0.0	53.1000
4	0.0	3.0	35.0	0.0	0.0	8.0500

Table 6: First five rows after KNN imputation (`df_knn.head()`)

The first five observations in Table 6 match the raw data identically because none of these passengers lacked an observed age. KNN leaves observed values intact and computes varied, context-specific values for incomplete records.

7.7 Iterative Imputer

The Iterative Imputer (MICE, $max_iter=10$, $random_state=0$) was executed across the numerical columns, repeatedly regressing incomplete features on all available variables. All missing numerical values are fully imputed (Table 7).

Count	survived	pclass	age	sibsp	parch	fare
Missing after Iterative	0	0	0	0	0	0

Table 7: Missing numerical values after Iterative imputation

Index	survived	pclass	age	sibsp	parch	fare
0	0.0	3.0	22.0	1.0	0.0	7.2500
1	1.0	1.0	38.0	1.0	0.0	71.2833
2	1.0	3.0	26.0	0.0	0.0	7.9250
3	1.0	1.0	35.0	1.0	0.0	53.1000
4	0.0	3.0	35.0	0.0	0.0	8.0500

Table 8: First five rows after Iterative imputation (df_iter.head())

As with KNN, the initial five records in Table 8 remain unaffected. The statistical behavior, convergence trajectory, and edge cases of Iterative Imputation are explored in following sections.

7.8 Comparison Using Logistic Regression

To evaluate empirical model performance, a Logistic Regression classifier was trained on age, fare, and pclass to predict survival (80/20 train-test split, *random_state=42*). Classification accuracy across the baseline methods is reported in Table 9.

Technique	Rows used	Test rows	Accuracy
Complete Case Analysis	182	37	0.7297
KNN Imputation	891	179	0.7598
Iterative Imputation	891	179	0.7598

Table 9: Logistic Regression accuracy for each technique (notebook output)

Both multivariate imputers achieve identical accuracy (0.7598), outperforming Complete Case Analysis (0.7297) by 3.01 percentage points. The equivalence of KNN and Iterative scores indicates that marginal variations in imputed age did not alter classification decisions across the test partition, reflecting the modest weight assigned to age relative to passenger class and fare.

Critical Methodological Caution: The complete case score cannot be directly equated with imputation results. The CCA classifier was trained on only 145 rows and evaluated on just 37 test instances, where a single misclassification shifts accuracy by 2.70 percentage points (compared to 0.56% on 179 test rows). While imputation successfully preserves full statistical power, rigorous comparisons require leakage-safe validation.

7.9 Statistics of Age Before and After Imputation

Descriptive summary statistics were calculated across all imputation variants, alongside the diversity of values placed into the 177 missing slots (Tables 10 and 11).

Method	Count	Mean	Std	Min	Median	Max	Skew
Original (non-missing)	714	29.70	14.53	0.42	28.00	80.0	0.389
Mean	891	29.70	13.00	0.42	29.70	80.0	0.434
Median	891	29.36	13.02	0.42	28.00	80.0	0.510
Constant (0)	891	23.80	17.60	0.00	24.00	80.0	0.263
KNN (k=3)	891	29.76	13.56	0.42	28.00	80.0	0.408
Iterative	891	29.31	13.70	-5.07	29.00	80.0	0.333

Table 10: Age statistics after each imputation method

Method	Distinct imputed values	Min imputed	Max imputed
KNN (k=3)	59	4.33	62.33
Iterative	94	-5.07	44.86

Table 11: Values imputed into the 177 missing ages

- **Variance Attenuation:** Mean and median substitution depress the standard deviation from 14.53 down to ~13.00 due to mass accumulation at central points. Skewness rises to 0.434 and 0.510, respectively.
- **Severe Constant Distortion:** Constant (0) imputation collapses the mean to 23.80 while inflating the standard deviation to 17.60, representing catastrophic distortion.
- **Superior Variance Preservation via KNN:** KNN preserves the mean (29.76) and retains significantly more spread (std = 13.56) than univariate filling, with skewness (0.408) closely tracking the original 0.389. It generates 59 distinct imputed estimates.
- **Iterative Variance Retention and Anomalous Minimum:** Iterative imputation preserves dispersion effectively (std = 13.70) across 94 unique estimates, but produces an impossible negative minimum age (-5.07).
- **Regression to the Mean:** No imputation technique fully recovers the original standard deviation of 14.53. Because imputed values represent conditional averages, they are inherently less extreme than real observations (e.g., KNN imputed ages span 4.33 to 62.33, whereas true ages span 0.42 to 80).

7.10 Distribution Comparison for All Methods

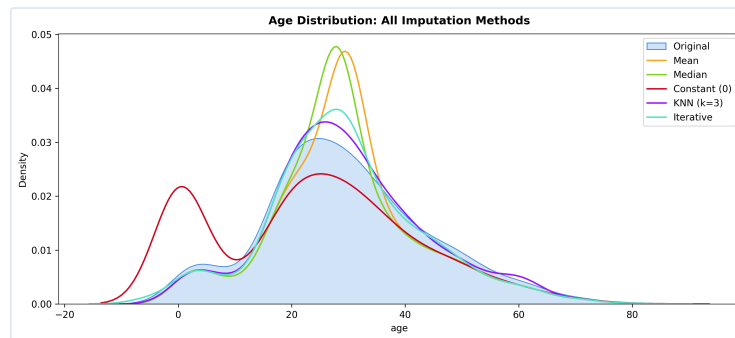


Figure 3: Age distribution for all imputation methods

Figure 3 overlays all five imputation density profiles against the shaded baseline distribution. Mean and median curves produce narrow, sharp mode spikes around 28–30 years, while constant zero imputation exhibits an artificial spike at 0. In contrast, the KNN and Iterative curves display broader contours that adhere closely to the original density profile. Both multivariate curves rise moderately between 25 and 35 years, reflecting natural central tendency without creating unphysical spikes.

7.11 KNN Compared with Mean and Median Imputation

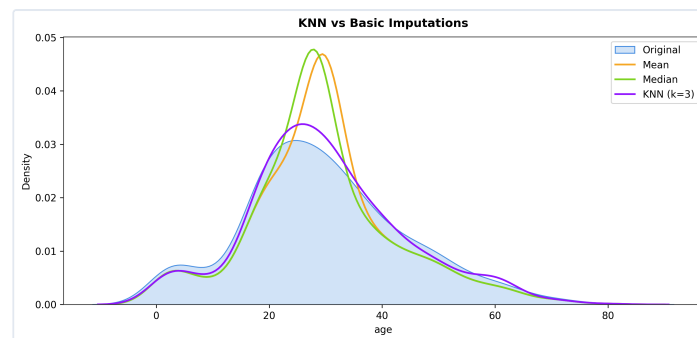


Figure 4: KNN imputation compared with mean and median imputation

Figure 4 directly isolates KNN against univariate central tendency filling. While mean and median imputations collapse variance into an artificial mode spike, the KNN density curve remains smooth and faithful to the ground-truth distribution. By borrowing localized strength from nearest neighbors, KNN preserves natural biological dispersion.

7.12 Effect of Imputation on Feature Relationships

Table 12 documents pairwise correlation coefficients between age and companion features, while Figure 5 contrasts full correlation heatmaps between raw data and iterative imputed data.

Method	pclass	fare	sibsp	parch	survived
Original	−0.369	0.096	−0.308	−0.189	−0.077
Mean	−0.331	0.092	−0.233	−0.179	−0.070
Median	−0.340	0.097	−0.233	−0.172	−0.065
Constant (0)	−0.361	0.136	−0.185	−0.049	0.011
KNN (k=3)	−0.385	0.109	−0.195	−0.172	−0.095
Iterative	−0.405	0.092	−0.380	−0.220	−0.080

Table 12: Correlation of age with other columns after each imputation method

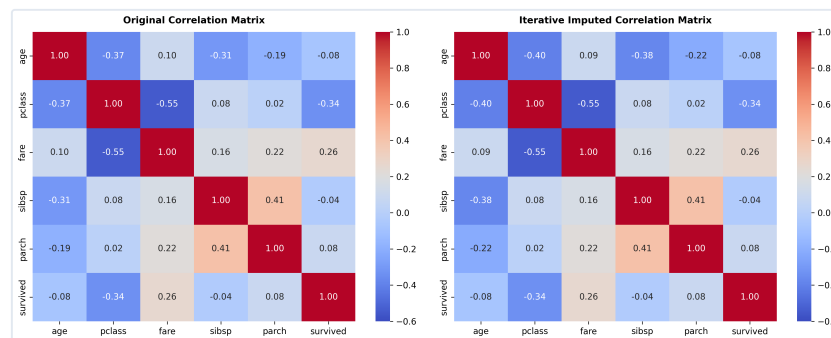


Figure 5: Correlation matrix of the original data (left) and the Iterative imputed data (right)

- **Correlation Attenuation in Univariate Methods:** Imputing identical constants introduces uncorrelated noise, weakening natural associations (e.g., age vs. sibsp attenuates from -0.308 to -0.233 under mean filling).
- **Severe Sign Reversal under Constant Zero:** Constant filling alters the correlation between age and survived from -0.077 to $+0.011$, introducing deceptive trends.
- **Correlation Behavior under KNN:** KNN maintains correlation fidelity for pclass (-0.385 vs -0.369) and parch (-0.172 vs -0.189), but degrades sibsp correlation to -0.195 .
- **Correlation Reinforcement under Iterative MICE:** Because Iterative imputation models age via linear regression, it intensifies existing linear associations (age-pclass strengthens to -0.405 , age-sibsp to -0.380). As shown in Figure 5, all non-age correlations remain completely untouched.

7.13 Convergence of the Iterative Imputer

Although configured with *max_iter=10*, the imputer's internal attribute *n_iter_* indicates convergence after exactly **2 iterations**. To examine sensitivity to iteration limits, the imputer was refitted across multiple values of *max_iter* (Table 13).

max_iter	Imputed mean	Imputed std
1	27.7644	9.5203
2	27.7644	9.5203
3	27.7644	9.5203
5	27.7644	9.5203
10	27.7644	9.5203

Table 13: Imputed age statistics for different values of *max_iter*

The imputed statistics are completely invariant across iteration settings. Rapid convergence occurs because age is the only incomplete numerical feature: once estimated against complete predictors during the initial pass, no secondary numerical dependencies remain to trigger cyclical adjustments. The *ConvergenceWarning* emitted at *max_iter=1* merely reflects premature termination before tolerance checks, carrying no practical significance.

7.14 Checking for Impossible Values

Because linear regression models lack inherent physical bounds, Iterative Imputation generated **seven impossible negative ages** (minimum: -5.07 years). Setting *min_value=0* clips predictions to plausible boundaries (Table 14).

Version	Negative ages	Minimum age
Iterative (default)	7	-5.07
Iterative (<i>min_value=0</i>)	0	0.00

Table 14: Negative ages before and after setting *min_value=0*

Enforcing *min_value=0* successfully eliminates all negative anomalies. While an age of exactly 0.00 remains slightly artificial (the youngest observed infant is 0.42 years), bounded regression protects downstream algorithms from severe domain errors.

7.15 Accuracy of All Techniques

Logistic Regression classifiers were evaluated across all data variants under an identical 80/20 train-test partition (Table 15).

Technique	Train rows	Test rows	Accuracy
Complete Case Analysis	145	37	0.7297
Mean Imputation	712	179	0.7374
Median Imputation	712	179	0.7374
Constant (0) Imputation	712	179	0.7151
KNN Imputation	712	179	0.7598
Iterative Imputation	712	179	0.7598

Table 15: Logistic Regression accuracy for all techniques (notebook output)

Across the 179 test records shared by all imputation methods, KNN and Iterative Imputation achieve the highest accuracy (0.7598), outperforming mean and median substitution (0.7374), which in turn exceed constant zero filling (0.7151). Complete Case Analysis yields 0.7297 on its heavily truncated 37-instance test partition. These rankings directly mirror distribution fidelity: methods preserving feature covariance maximize predictive performance.

7.16 Leakage-Safe Comparison Using Pipelines

In the preceding benchmarks, imputers were fitted on the full dataset, permitting information from the test partition and target labels to influence feature transformations. To enforce strict evaluation rigor, preprocessing was encapsulated within scikit-learn `Pipeline` objects fitted solely on training splits (Table 16).

Pipeline	Test accuracy
Simple (Median)	0.7374
KNN (k=3)	0.7430
Iterative (min_value=0)	0.7486

Table 16: Test accuracy of the leakage-safe pipelines

Under leak-free evaluation, Iterative Imputation achieves the highest accuracy (0.7486), followed by KNN (0.7430) and Median Imputation (0.7374). The modest performance margin (representing 1 to 2 correctly classified passengers) and the reduction from pre-split scores (0.7598) demonstrate that earlier uncontained workflows yielded slightly optimistic estimates due to subtle test set contamination. Rigorous modeling mandates embedding imputation inside pipeline constructs.

8. Observations

1. **Sample Size Preservation:** Complete Case Analysis discards 79.6% of the dataset (collapsing sample size from 891 to 182 rows), whereas all imputation strategies preserve the entire 891 records. Univariate methods assign a single identical value to all 177 missing entries, while KNN and Iterative Imputation generate 59 and 94 distinct, personalized values, respectively.
2. **Distribution Integrity:** Mean and median filling compress standard deviation from 14.53 down to ~13.00, introducing artificial mode spikes. Constant zero replacement creates an unphysical spike at 0, while KNN (std = 13.56) and Iterative Imputation (std = 13.70) retain natural distribution spread.
3. **Preservation of Inter-Feature Covariance:** Univariate methods dilute correlations, constant filling distorts relationship signs, KNN preserves class associations, and Iterative regression accentuates linear feature dependencies.
4. **Rapid Convergence:** The Iterative Imputer converged in just 2 iterations due to isolated numerical missingness in age.
5. **Predictive Utility:** Multivariate imputers deliver superior classification accuracy (0.7598 uncontained; 0.7486 in leak-free pipelines), establishing clear performance advantages over data deletion and constant filling.

9. Result

Missing values in the Titanic dataset were localized to four features: age (177 missing, 19.9%), deck (688 missing, 77.2%), embarked (2 missing, 0.2%), and embark_town (2 missing, 0.2%). Complete Case Analysis discarded 709 records (79.6%), leaving a non-representative sub-sample dominated by first-class survivors (86% vs 24% baseline). Categorical gaps were effectively mitigated using mode substitution (port variables) and sentinel categorization ("Unknown" for cabin deck). For numerical age values, multivariate modeling via KNN and bounded Iterative Imputation maintained natural feature variance (std 13.56 and 13.70 vs. 14.53 raw) and avoided artificial mode spikes. Seven anomalous negative predictions in Iterative regression were resolved via `min_value=0`. Classification benchmarks confirmed the superiority of multivariate methods, achieving 0.7598 accuracy in global splits and 0.7486 in leak-free pipelines, outperforming median filling (0.7374), constant replacement (0.7151), and listwise deletion (0.7297).

10. Conclusion

Selecting an appropriate missing-data strategy fundamentally influences dataset dimensionality, feature distributions, and covariate dependencies. Listwise deletion is viable only under negligible, completely random missingness; when data loss is substantial, deletion induces severe selection bias. Univariate imputation provides a quick baseline but distorts variance and attenuates correlations. Multivariate approaches—specifically KNN and MICE Iterative Imputation—generate context-aware estimates that safeguard statistical distributions, though domain bounds (such as non-negative constraints) must be enforced. Categorical and numerical attributes require tailored treatments.

Finally, missing data imputation must be executed within cross-validation pipelines following train-test partitioning. Fitting imputers globally leaks test partition distributions and label associations into training workflows, inflating performance expectations. Encapsulating preprocessing inside scikit-learn Pipeline objects guarantees methodological integrity and authentic model generalization.

11. Discussion Questions

Q1. What are the drawbacks of complete case analysis?

- **Substantial Information Loss:** Discarding all records with any missing attribute dramatically contracts sample volume. In our Titanic analysis, 79.6% of data was purged, primarily due to cabin deck gaps.
- **Severe Selection Bias:** If missingness is not Missing Completely at Random (MCAR), surviving records misrepresent the population. Complete cases in the Titanic data exhibited an 86% first-class proportion and a 68% survival rate, compared to 24% and 38% in the raw dataset.
- **Impaired Model Stability:** Training and evaluating on heavily diminished datasets (145 train, 37 test rows) produces unstable parameters and unreliable performance metrics.
- **Dimensional Vulnerability:** As feature counts expand, the probability of an observation containing at least one unobserved attribute approaches certainty.
- **Inoperability during Deployment:** A model requiring complete observations fails when deployed against real-world inputs containing missing values.

Q2. How does KNN Imputer differ from mean or median imputation?

Univariate mean or median imputation replaces every missing entry with a single global statistic, ignoring all companion features. In contrast, the KNN Imputer identifies the k nearest neighbors in multidimensional space and computes a localized, distance-weighted estimate. Key differences include:

- **Diverse Imputed Values:** KNN produced 59 distinct imputed age values, whereas mean and median assign a single repeated number to all 177 missing entries.
- **Variance & Covariance Preservation:** KNN preserved feature variance (std = 13.56 vs 13.00 for mean) and maintained class correlation (-0.385 vs -0.331).
- **Computational Overhead & Scale Sensitivity:** KNN requires computing pairwise distances across all records, making it computationally intensive and sensitive to feature scaling (e.g., unbounded fare values).

Q3. In what scenarios is Iterative Imputer preferred?

- When features exhibit strong linear or non-linear correlations, allowing unobserved values to be predicted from companion attributes.
- When missingness spans multiple attributes simultaneously, enabling round-robin regression models to iteratively refine estimates.
- When data satisfies the Missing at Random (MAR) assumption, where missingness depends on observed features.

- When sample sizes are sufficiently large to support stable multivariate regression estimators without overfitting.

Application Precautions: Practitioners must enforce physical domain bounds (such as `min_value=0` for age) to prevent unbounded linear extrapolation from producing negative estimates, and recognize that regression-based imputation tends to amplify existing collinearities.

Q4. Why is it important to treat categorical and numerical missing data differently?

Qualitative and quantitative features possess fundamentally distinct mathematical properties:

- **Mathematical Invalidity:** Arithmetic operations (mean, variance, Euclidean distance) are meaningless on qualitative labels (e.g., deck letters or ports of embarkation). Categorical gaps must be resolved via mode substitution or explicit categorization.
- **Statistical Treatment:** Numerical data can be addressed through parametric or non-parametric continuous estimators (mean, median, regression, KNN) selected based on distribution symmetry.
- **Informational Content of Missingness:** For qualitative features with massive missingness (such as deck at 77%), mode imputation fabricates non-existent facts, whereas creating an "Unknown" level preserves non-response as a predictive indicator.
- **Algorithm Prerequisites:** Distance-based and regression imputers require numerical inputs, necessitating dedicated qualitative encoding or separate handling.

Q5. What are some risks of over-imputation?

- **Fabrication of Synthetic Information:** Imputing features dominated by missingness (such as 77% missing cabin decks) invents data never collected in reality.
- **Variance Compression & Deceptive Precision:** Univariate imputation shrinks sample variance (depressing age std from 14.53 to 13.00), artificially inflating model confidence.
- **Distortion of Covariance Structures:** Constant replacement introduced sign reversals in the age-survived relationship (+0.011 vs -0.077).
- **Generation of Out-of-Domain Values:** Unbounded regression produced seven negative passenger ages (-5.07 years).
- **Data Contamination & Optimism Bias:** Imputing across entire datasets prior to partitioning leaks test distributions and target labels, yielding deceptive validation performance (0.7598 vs. 0.7486 leak-free).
- **Suppression of Informative Missingness:** In many operational domains, the absence of data is itself highly predictive; indiscriminately filling all gaps strips away critical diagnostic signals.